Unsupervised Learning

Lesson 8 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

13 November 2026 · reading time about 95 minutes

Contents

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 7 returned; what changes when there is no y	Slides 2–6
0:10–0:35	25	k-means: the objective, Lloyd's algorithm, k-means++, choosing k	Slides 7–23
0:35–0:55	20	**Notebook 01** — k-means from scratch	Slide 24
0:55–1:10	15	When round clusters are the wrong assumption: hierarchical clustering	Slides 25–31
1:10–1:22	12	**Break**	Slide 32
1:22–1:40	18	DBSCAN: density instead of shape, and whether `eps` transfers	Slides 33–40
1:40–2:00	20	**Notebook 02** — hierarchical clustering and DBSCAN	Slide 41
2:00–2:10	10	Validating a clustering with no labels to check against	Slides 42–45
2:10–2:35	25	Principal component analysis (PCA), derived twice: eigendecomposition and the singular value decomposition (SVD)	Slides 46–58
2:35–2:55	20	**Notebook 03** — PCA, anomaly detection, t-SNE	Slide 59
2:55–3:00	5	t-SNE in one slide; homework	Slides 60–62
	180	**Total**	**61 slides, 3 notebooks**

---

1. Why this lesson exists

Every method in this course so far has learned from pairs: an input x and a target y — a price, a class, a default or not. **Unsupervised learning** removes y entirely. What is left is a table of measurements with no answer key, and the task is no longer "predict the label" but "describe the structure" — which points look alike, which few numbers summarise the many, which points do not

belong.

This matters practically as much as it matters conceptually. Most data an organisation collects was never labelled by anyone, and labelling it is expensive, slow, or — for a question like "which of these customers are alike" — not even well-defined until someone runs a clustering and looks. Unsupervised methods are frequently the *first* thing run on a new dataset, supervised learning the second, once the first pass has suggested what the targets or the useful features even are.

1.1 One retailer, three problems

This lesson follows **Aurora**, a fictional online retailer, through three questions, one per notebook, that share nothing except that none of them has a y:

1. **Segment 2,000 customers** for a marketing campaign, from two numbers per customer — annual spend and visit frequency — with no segment labels supplied by anyone (**k-means**, notebook 01).
2. **Tell bots from humans** in 1,500 weekly browsing sessions, where the two groups share the same *average* behaviour and differ only in how consistent that behaviour is (**hierarchical clustering and DBSCAN**, notebook 02).
3. **Compress an 8-column account table down to what actually varies, and catch a handful of fraudulent accounts hiding in it** — accounts that are unremarkable on every single column and give themselves away only in how the columns relate to each other (**PCA and reconstruction error**, notebook 03).

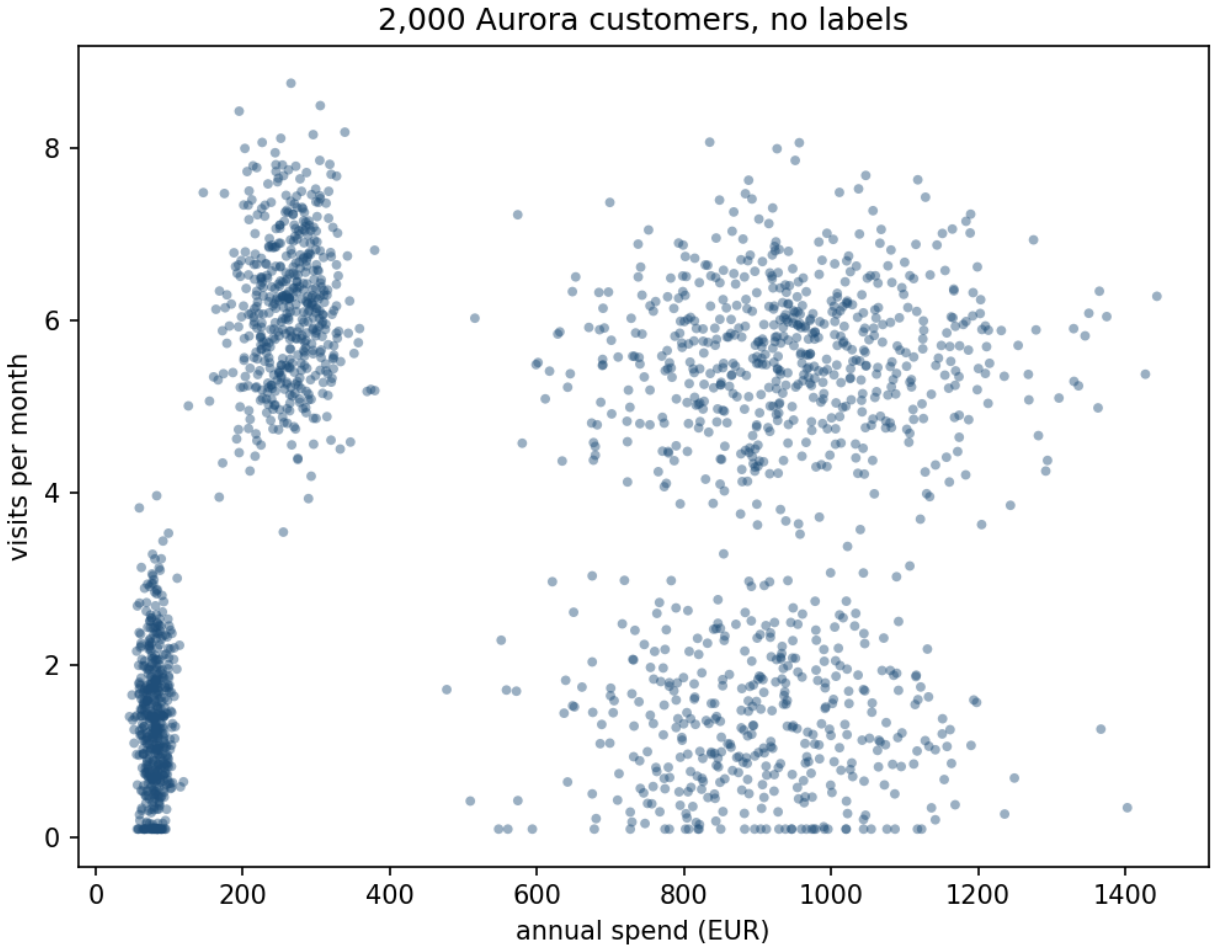
Every dataset in this lesson is synthetic, and — unusually for the real version of any of these problems — the *true* generating structure is published in `Notebooks/retail_data.py` as `TRUE_*` constants, precisely so that each method's answer can be checked against a ground truth that Aurora's own analysts would never actually have. That asymmetry is worth holding onto through the whole lesson: it is what makes this lesson possible to teach, and it is exactly what will be missing the first time any of these methods runs on real data.

> **Try this:** before reading any further, open `Notebooks/retail_data.py` and read the docstring of `make_customer_segments`. Try to predict, from the four segments' means alone, roughly where each one will sit on a scatter plot of spend against visit frequency — then check section 2's figure.

---

2. k-means: the objective, and why it converges

This is what Aurora's marketing team actually has: two numbers per customer — annual spend and visits per month — with no group name attached to either.



2,000 Aurora customers, no labels. Four clouds are visible to the eye before any algorithm runs, which is not an accident of this dataset: it is exactly the structure k-means is built to exploit.

2.1 The objective, and Lloyd's algorithm

The intuition first, with no symbols: if customers really do fall into a handful of natural groups, a good grouping is one where every customer sits close to a single representative point for their group — the group's "typical customer" — and far from the representative points of every other group. k-means turns "close" into a number and searches directly for the grouping that makes the total closeness as large as possible.

Formally, k-means seeks the assignment of every point $x_i \in \mathbb{R}^n$ to one of k clusters, and the k cluster centres $\mu_1, \dots, \mu_k$, that jointly minimise the **within-cluster sum of squares**:

$$J(\{\mu_j\}, \{r_{ij}\}) = \sum_{i=1}^{m} \sum_{j=1}^{k} r_{ij} \, \|x_i - \mu_j\|^2, \qquad r_{ij} \in \{0, 1\}, \quad \sum_{j=1}^{k} r_{ij} = 1$$

where $r_{ij} = 1$ exactly when point i is assigned to cluster j. J is a function of two kinds of variables at once — the assignments r_{ij} and the centres μ_j — and minimising over both jointly is computationally hard: there are more ways to partition m points into k groups than any search can enumerate once

m passes a few dozen. What makes the problem tractable is that **minimising over each kind of variable separately, holding the other fixed, has a closed-form answer.**

Fixing the centres, minimise over the assignment. For each point i independently, J is a sum over j of the (non-negative) term $r_{ij}\|x_i - \mu_j\|^2$ with exactly one r_{ij} allowed to be 1. The minimum is achieved by setting $r_{ij} = 1$ for whichever j minimises $\|x_i - \mu_j\|^2$ — assign every point to its **nearest centre**.

Fixing the assignment, minimise over the centres. For each cluster j independently, J restricted to cluster j's points is $\sum_{i:r_{ij}=1}\|x_i - \mu_j\|^2$, a convex quadratic in μ_j. Setting its gradient to zero,

$$\frac{\partial}{\partial \mu_j} \sum_{i:r_{ij}=1} \|x_i - \mu_j\|^2 = -2 \sum_{i:r_{ij}=1} (x_i - \mu_j) = 0 \quad \Longrightarrow \quad \mu_j = \frac{1}{n_j} \sum_{i:r_{ij}=1} x_i$$

the unique minimiser is the **mean** of the points currently assigned to cluster j — which is where the algorithm's name comes from.

Lloyd's algorithm alternates these two exact minimisations: assign, then update, then assign again, until nothing changes. Each step, by construction, cannot increase J — it is solving an exact minimisation of J over the variables it controls, so the new value is at most the old one. J is bounded below by 0, and there are only finitely many ways to partition m points into k groups, so the (non-increasing) sequence of J values cannot decrease forever: it must reach a partition that repeats, at which point neither step changes anything and the algorithm has converged. What it converges *to* is a **local** minimum of J — a partition no single reassignment or recentring can improve — not necessarily the global one, which is exactly the problem section 2.2 demonstrates.

Worked example. Take six standardised customers from the dataset (the notebook's first six rows) and two badly chosen initial centres — the first two points themselves:

Iteration	Assignment	WCSS after assign step	WCSS after update step
0	`[0 1 1 1 0 1]`	15.965	7.281
1	`[0 0 1 1 0 1]`	5.491	4.674
2	`[0 0 1 1 0 1]`	4.674	4.674

J falls at every single step — 15.965 → 7.281 → 5.491 → 4.674 — and by iteration 2 the assignment repeats and both steps leave J unchanged: the algorithm has converged, in this case to the global optimum for these six points. `Docs/worked_examples.py` reproduces this table from `retail_data.py` directly.

2.2 Initialisation: a bad start is a real risk

Lloyd's algorithm needs a starting set of k centres before it can take its first step, and *which* local minimum it reaches depends entirely on where those centres start. The naive choice — k points picked uniformly at random from the data — can start the search close to the global optimum, or nowhere near it, with no way to tell in advance which.

On the 2,000-customer dataset, running Lloyd's algorithm from 30 independent naive random starts gave a best final WCSS of **374.1** and a worst of **1,390.4** — the worst run left the algorithm stuck in a local minimum **3.7 times worse** than the best, using the identical update rule on the identical data. This is not a rare pathology; it happened on the very first attempt (worst of 30, not one in a thousand).

k-means++ addresses this directly at initialisation time, before Lloyd's algorithm ever runs: the first centre is a uniformly random point, and every subsequent centre is chosen with probability proportional to its squared distance from the *nearest centre already chosen*:

$$P(x_i \text{ chosen next}) = \frac{d(x_i)^2}{\sum_{i'} d(x_{i'})^2}, \qquad d(x_i) = \min_{j \text{ chosen}} \|x_i - \mu_j\|$$

A point already close to an existing centre is unlikely to be picked again; a point far from every existing centre — plausibly the seed of a cluster not yet represented — is favoured. Across the same 30 seeds, k-means++ starts ranged from 374.1 to 1,088.3, narrowing the worst case by roughly 300 units and, more importantly, reaching the global optimum on the very first attempt with default settings — which is why `sklearn.cluster.KMeans` uses k-means++ by default, together with 10 independent restarts, keeping the best.

The from-scratch implementation in notebook 01, run once from a k-means++ start, matches scikit-learn's 10-restart result to within 10^{-4} of WCSS — not a coincidence, since both are searching for the same optimum from starts drawn the same way, but a genuine check rather than an assumption.

2.3 Choosing k: the elbow and the silhouette

Nothing said so far determines k — it is an input to the algorithm, not something it discovers. Two diagnostics are standard, and they measure different things, which is why using both is worth the small extra cost.

The elbow. WCSS is monotonically non-increasing in k — with $k = m$ every point is its own cluster and $J = 0$ exactly — so the raw value of J at the best k is not the signal. What is informative is *where the curve stops falling steeply*: past the true number of clusters, adding another one only subdivides a cluster that was already coherent, buying a small reduction in J for the cost of a whole extra centre.

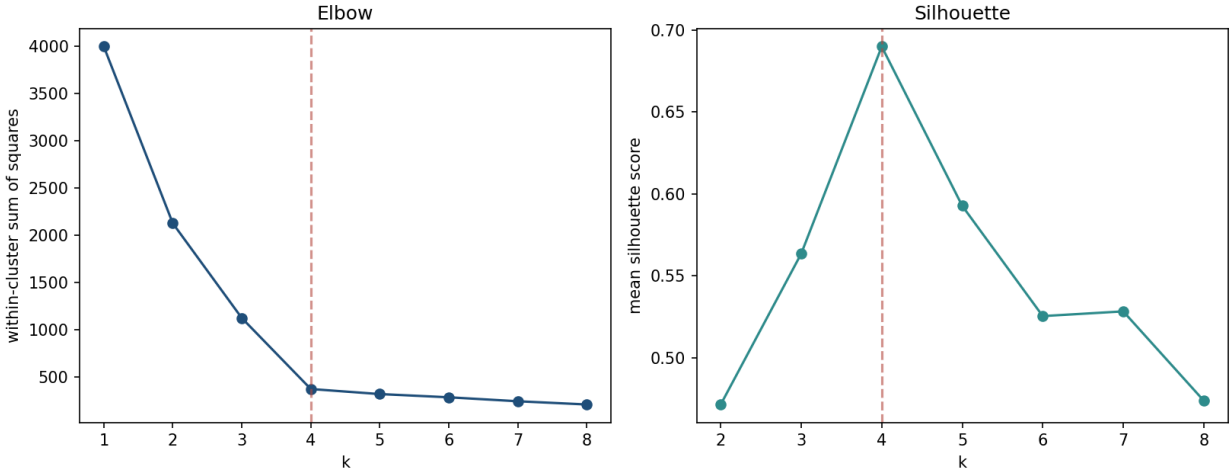
The silhouette score. For point i, let a_i be its mean distance to every other point in its own cluster, and b_i its mean distance to the points of the *nearest other* cluster. The silhouette is

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$$

$s_i \to 1$ when a point sits deep inside its own cluster and far from every other; $s_i \approx 0$ on a boundary between two clusters; $s_i < 0$ when a point is, on average, *closer* to a different cluster than its own — an assignment the silhouette treats as actively wrong, not merely ambiguous. Averaged over all points, it gives a single number per k that is a genuine optimum to search over, not only a shape to eyeball.

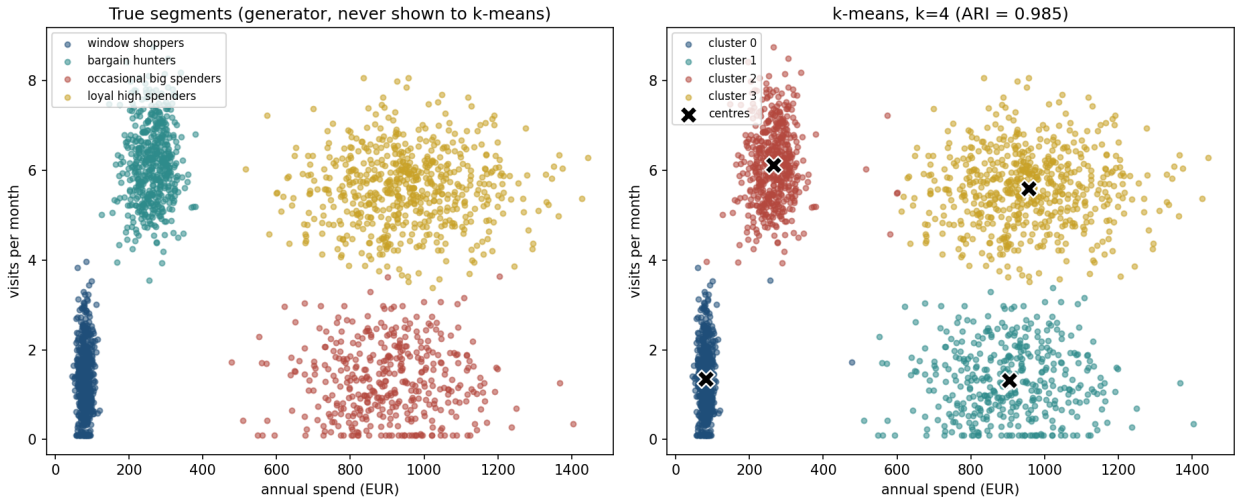
k	WCSS	Silhouette
1	4000.0	—
2	2128.5	0.472
3	1125.8	0.564
4	**374.1**	**0.690**
5	320.6	0.593
6	286.5	0.525
7	244.5	0.528
8	210.4	0.474

The elbow and silhouette curves on the customer-segment data. WCSS falls sharply through $k = 4$ and flattens after; silhouette peaks at $k = 4$ before declining. Section 4 works through a dataset where the two disagree.



Two different questions, one dashed line. Left: the within-cluster sum of squares stops falling steeply after $k = 4$, which is the "elbow" — note how much judgement reading that bend takes. Right: the mean silhouette has an unambiguous maximum at the same k, which is why the second diagnostic is the more useful one when they disagree.

Both diagnostics agree here, and agree with the number of segments the data was actually built with — a agreement worth noting precisely because it does not always happen, and is not required to. WCSS asks only "does adding a cluster help," a question that is agnostic to what a cluster *means*; silhouette asks whether every point is closer to its own group than to any rival, which is a stronger and more geometric condition. That the two agree here is a property of this particular dataset's four well-separated, comparably sized blobs, not a guarantee.



Left: the true segments, generated by `retail_data.py` *and never shown to k-means. Right: k-means' own* $k = 4$ *clustering, with the fitted centres marked. The adjusted Rand index between the two is 0.985 — near-perfect agreement — computed only because this lesson knows the truth; Aurora's own analysts would have the right panel and nothing to check it against.*

The **adjusted Rand index** (ARI) used to score that agreement compares two labellings of the same points, corrected so a random assignment scores 0 and identical labellings score 1 (up to a relabelling of which cluster is called which — cluster "0" need not correspond to the same segment name on both sides). It is an **external** validation metric: it requires a ground truth, which section 4 returns to.

> **Try this:** run k-means with $k = 3$ on the customer-segment data and look at which two of the four true segments get merged into one cluster. Given the four segment means in `retail_data.py`, which pair would you predict merges first, and why?

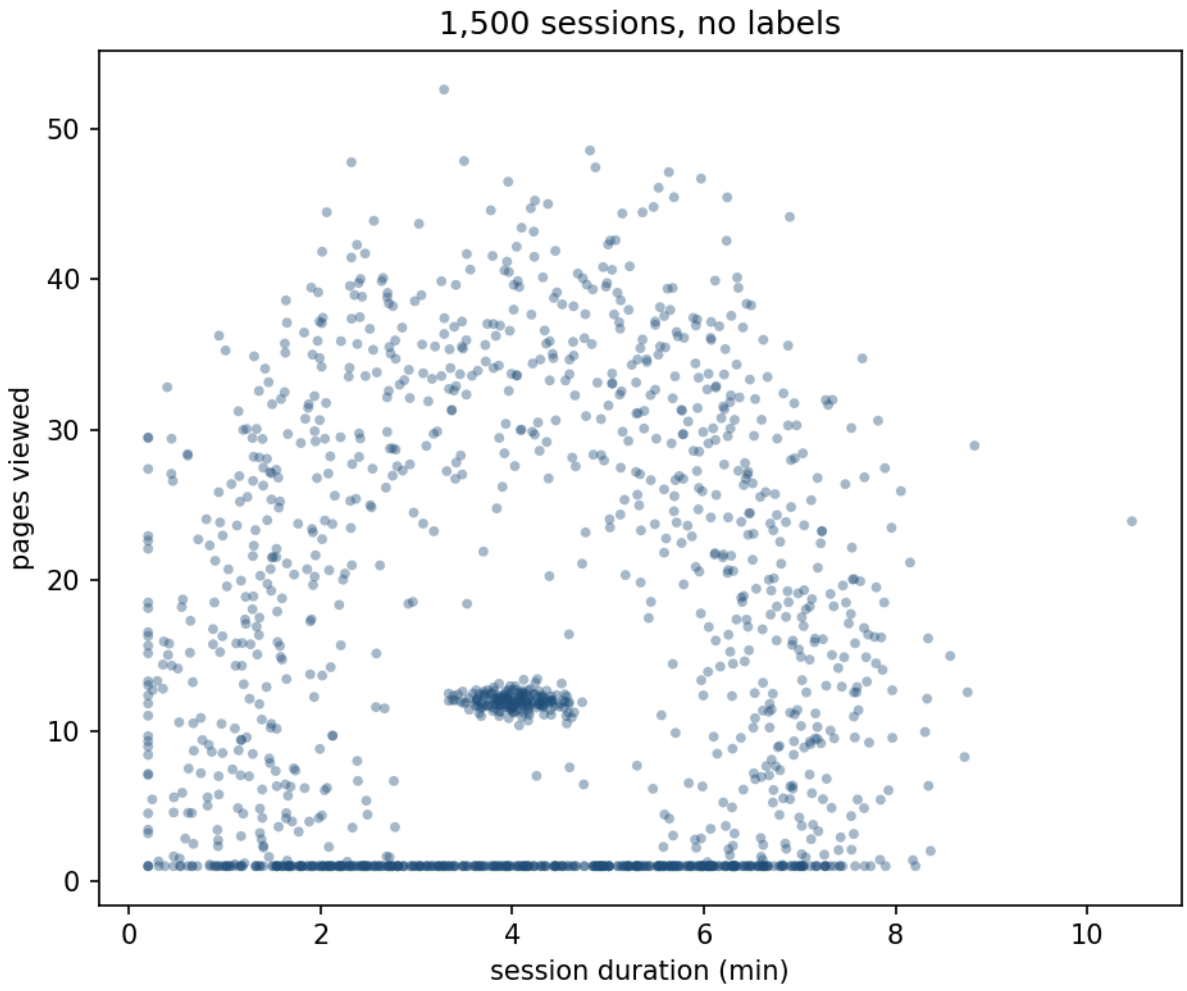
---

3. When round clusters are the wrong assumption

3.1 A dataset built to defeat k-means

Aurora's security team suspects that some fraction of last week's sessions were automated. Two features are logged per session — duration in minutes, and pages viewed — and 12% of the 1,500 sessions are, by construction, **bots**: scripted, and therefore behaviourally consistent from one session to the next. The other 88% are genuine humans, whose browsing is far more variable.

Crucially, the two groups are built to share the **same mean** duration and page count — bots are a tight cloud centred at (4.0 min, 12 pages); humans are spread into a ring *around that same centre*, not a separate blob beside it. This is deliberate: it removes the option of separating the groups with anything resembling a straight cut through the middle of two offset clouds, the shape every method in section 2 is built to find.



1,500 sessions, unlabelled — a dense smudge sitting inside a much larger, looser cloud, both centred in roughly the same place. There is no gap a linear or a round boundary can exploit.

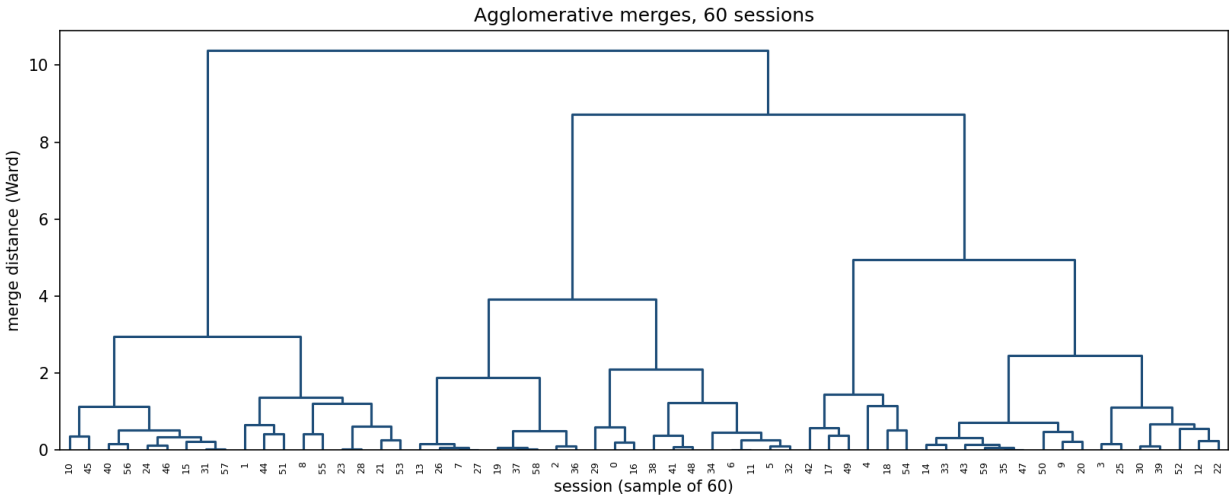
Run k-means with $k = 2$ on this data and the result is close to useless: **ARI** $= -0.046$, statistically indistinguishable from a random split of the same two group sizes. This is not a failure of tuning or initialisation — no choice of starting point fixes it, because WCSS is minimised by round, compact clusters, and the only way to cut this cloud into two round pieces is a line straight through the centre, which slices *both* the bot core and the human ring in half rather than separating them.

3.2 Hierarchical (agglomerative) clustering

Agglomerative clustering takes a different approach entirely: start with every point as its own cluster, and repeatedly merge the two closest clusters, recording every merge as a branch in a tree — a **dendrogram**. Cutting the tree at a given height is equivalent to choosing k: draw a horizontal line, and the number of vertical branches it crosses is the number of clusters at that cut.

"Closest" needs a rule for the distance *between two clusters* of possibly many points each, not only between two points — the **linkage criterion** — and this choice determines what shape of cluster the method can find at all:

- **Single linkage**: the distance between the closest pair of points, one from each cluster. Because merging only needs *one* close pair, it can trace a thin, winding chain of points arbitrarily far apart end to end — and just as easily string together two otherwise separate groups that touch at a single bridging point, a failure called **chaining**.
- **Complete linkage**: the distance between the *farthest* pair. Reluctant to merge anything that would create a large-diameter cluster, so it favours compact, comparably sized groups.
- **Average linkage**: the mean distance over every cross-pair — a compromise between the two above.
- **Ward linkage**: merges whichever pair of clusters increases total within-cluster variance the least. This is the *same objective* k-means minimises, expressed as a greedy merge rule rather than an iterative update — and predictably inherits the same bias toward round clusters.



Ward-linkage merges on a random sample of 60 sessions. A branch peeling off single points one at a time, rather than splitting into two comparably sized halves, is the visual signature of chaining — visible here even at this small sample.

Cutting all four linkage rules at two clusters and checking each against the true bot/human split:

Linkage	ARI	Cluster sizes
Ward	−0.062	1,007 / 493
Complete	−0.101	1,253 / 247
Average	−0.001	1,499 / 1
Single	−0.001	1,499 / 1

Ward and complete linkage fail for exactly the reason k-means did — both still search for compact, comparably sized pieces, which this shape does not have. Single and average linkage instead produce one cluster of 1,499 points and a cluster of 1: not a discovery, but chaining — the algorithm absorbed almost the entire dataset along its densest path before running out of points to merge, leaving nothing resembling the bot/human split. **All four linkage rules score at or below zero ARI on this dataset.** Hierarchical clustering is not, by itself, a fix for the shape problem k-means has here; it needs a rule that looks for *density* rather than compactness or chained proximity, which is what section 3.3 introduces.

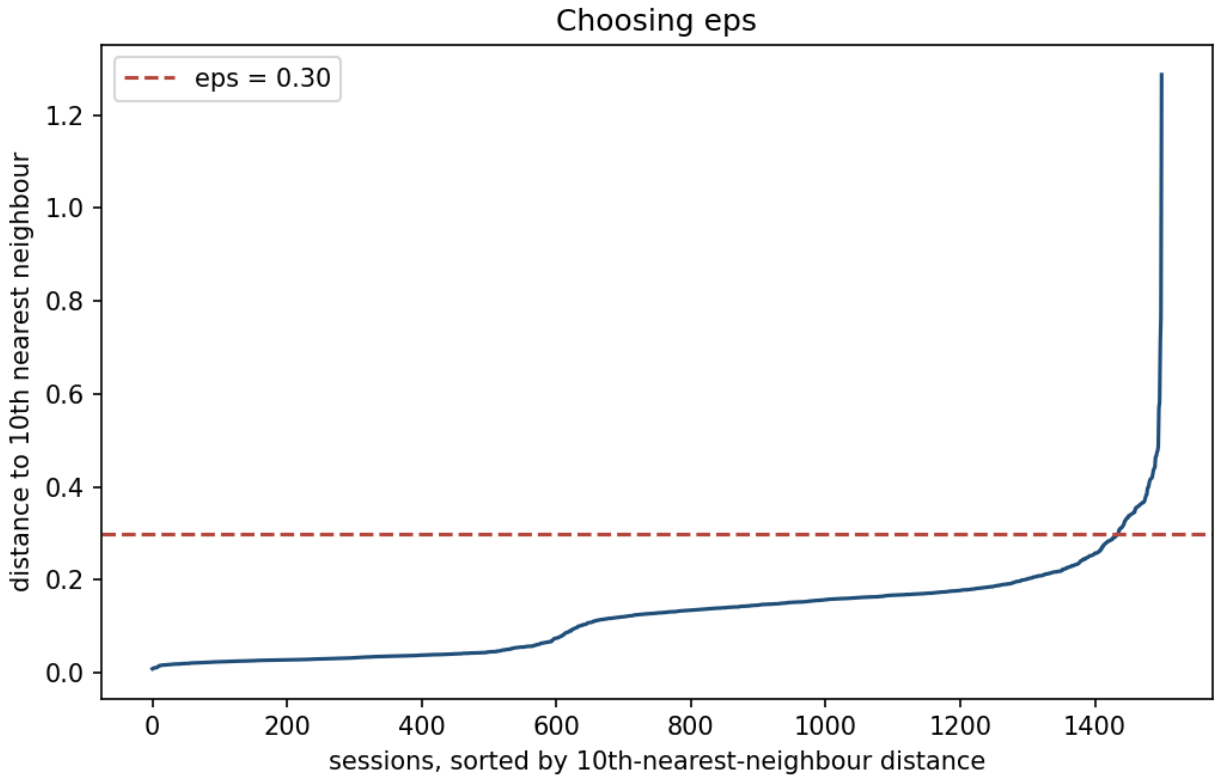
3.3 DBSCAN: density instead of shape

DBSCAN (density-based spatial clustering of applications with noise) classifies every point by how crowded its neighbourhood is, using two parameters: a radius `eps`, and a minimum count `min_samples`.

- A point is a **core point** if at least `min_samples` other points lie within distance `eps` of it.
- A point that is not itself a core point, but lies within `eps` of one, is a **border point** — it belongs to that core's cluster but cannot extend it further.
- Every other point is **noise**.

A cluster is a maximal set of core points connected to each other by chains of mutual `eps`-closeness, together with every border point attached to them. Nothing in this definition mentions a centre, a mean, or a shape — a cluster can be any connected region of sufficiently dense points, including a compact core *and*, separately, a diffuse ring, so long as the ring itself is dense enough at the scale `eps` measures.

Choosing `eps` is the parameter that matters most. Too small, and even the genuinely dense bot core fractures into isolated noise points; too large, and the ring's outer reaches bridge back across the gap into the core, merging everything into one cluster exactly as Ward linkage did. A ***k*-distance plot** — the distance from every point to its `min_samples`-th nearest neighbour, sorted from smallest to largest — gives a principled way to choose it: points in dense regions have a small such distance, points in sparse regions a large one, and the plot typically shows a sharp bend between the two regimes.

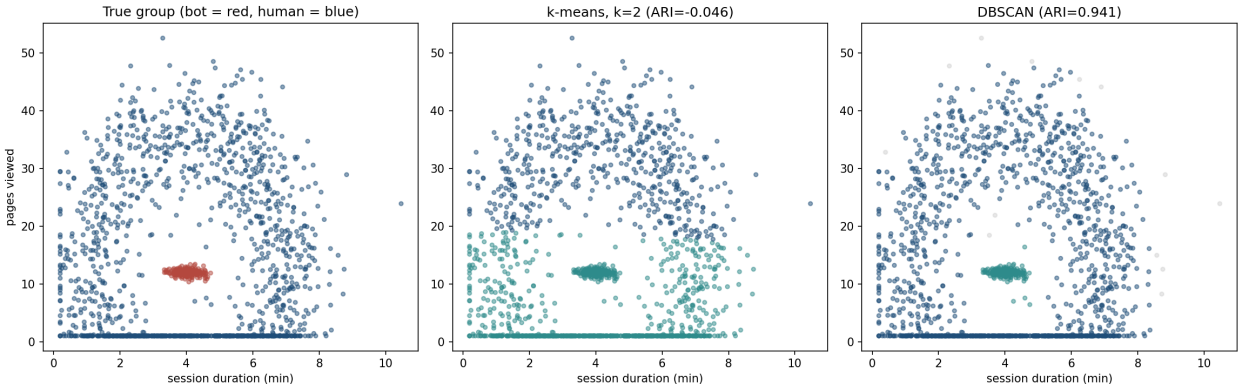


10th-nearest-neighbour distance, sorted, for all 1,500 sessions. The curve stays low and flat through most of the range and bends sharply upward near the end — `eps = 0.30` sits just past the bend, in

the flat region.

At `eps = 0.30`, `min_samples = 10`:

	Result
ARI against true bot/human split	**0.941**
Core points	1,435 of 1,500
Noise points	13



True group, k-means, and DBSCAN side by side on the same axes. DBSCAN's cluster boundary follows the actual density gap; k-means' straight cut does not exist in this data at all.

Every one of the 180 bot sessions lands in DBSCAN's dense-core cluster; 1,303 of the 1,320 human sessions land in the other; 4 humans are grouped with the bots; and 13 sessions are called noise. Those 13 are worth reading individually rather than discarding as an inconvenience — checking their true label shows **all 13 are genuine human sessions**, not misplaced bots. They are the customers whose one week of browsing happened, by chance, to look almost as mechanically regular as a script's. DBSCAN's answer for them — *not confidently either group* — is a more honest output than a forced binary label would be, and it is an answer no method in section 2 or 3.2 is even capable of giving, since every one of those methods must assign every point to exactly one cluster.

3.4 Does `eps` transfer?

`eps = 0.30` was read off *this* week's k-distance curve. Aurora's security team will run the same script next week, on a different week of sessions. Does the number come with it?

Notebook 02 answers that directly: regenerate the sessions at other seeds — same site, same bot fraction, same shapes, different individual sessions — and run both the fixed constant and the recipe that produced it.

Week	ARI at `eps = 0.30`	Clusters found	Knee `eps`	ARI at knee `eps`
this one	**0.941**	2	0.258	0.890
2	0.949	2	0.211	0.788
3	0.962	2	0.230	0.817
4	**−0.012**	**1**	0.226	0.858

Week	ARI at `eps = 0.30`	Clusters found	Knee `eps`	ARI at knee `eps`
5	**−0.009**	**1**	0.232	0.887
6	0.955	2	0.232	0.828
7	0.931	2	0.225	0.850

The constant does not transfer. On two of the seven weeks DBSCAN returns a single cluster and an ARI of essentially zero — not a worse answer, no answer at all. The mechanism is in the knee column: 0.30 is larger than any of these weeks' own curves suggest, all of which sit between 0.21 and 0.26. On five draws it is still small enough to keep the ring and the core apart; on two it is not, and the two merge into one component.

The recipe does transfer. Re-reading `eps` from each week's own curve recovers the split every time, at ARI 0.79 to 0.89.

And then the first row, which is the one worth sitting with. On this week the hand-picked 0.30 scores 0.941 against the recipe's 0.890. **The tuned constant is better where it works, and absent where it does not.**

That trade is not a fact about DBSCAN. It is what fitting a constant to one sample buys and costs in general: a number tuned to the data in front of you beats a rule that ignores it, right up until the data changes. Section 4 of lesson 5 made the same point about a single train/test split.

> **Try this:** re-run the sweep with `min_samples` at 5 and at 20. Does the fixed `eps = 0.30` become more fragile or less, and does the knee move with it?

3.5 Choosing among the three

Situation	Reach for
Clusters are plausibly round, similar in size, and k is known or can be searched	k-means — fast, and the objective is simple to reason about
You want the *hierarchy* itself — nested groupings at every scale, not one flat partition	Agglomerative clustering, linkage chosen for the expected shape
Clusters may be irregular, nested, or of very different densities, and outliers should be flagged rather than forced into a group	DBSCAN
The dataset is very large and every point must be assigned to something	k-means (DBSCAN's noise points need a separate policy)

The predictable mistake. Having watched k-means recover the customer segments almost perfectly in section 2, the reasonable conclusion is that clustering, as a category, works well. **The instinct is not wrong about k-means — it is wrong about clustering being one method with one assumption.** k-means and Ward linkage both optimise a notion of "cluster" that means *compact and round*; DBSCAN optimises a different notion, *densely connected*, and a nested structure like this lesson's bot core is exactly where the two notions give opposite answers. Which definition is the right one is a property of the data, not of the algorithm — which is why looking at the unlabelled scatter, in both section 2 and here, came before running anything.

---

4. Validating a clustering with no labels to check against

Every number checked so far — the ARI in sections 2.3 and 3 — used a ground truth this lesson happens to have because the data is synthetic. Aurora's actual analysts, on the actual customer or session data, will never have that. Two families of metric fill the gap, and it matters which one is available in a given situation.

Internal metrics use only the clustering and the data, never an outside answer key. The silhouette score from section 2.3 is one: it asks whether points are closer to their own cluster than to any other, a question answerable from the clustering alone. WCSS is another, though a weaker one, since it can only be compared across different values of k on the *same* algorithm, not across different clustering methods.

External metrics — ARI, and its relatives — compare a clustering against a separate labelling assumed to be correct. In this lesson that separate labelling is the data generator's `true_segment` or `true_group` column; in practice it might be a small hand-labelled sample, a business rule applied after the fact, or a downstream outcome (did the "bot" cluster's sessions actually get blocked by a rate limiter and never come back?). When such a signal exists — even a partial or noisy one — it should be used, because internal metrics can be confidently wrong: a silhouette score has no way to know that k-means' round-cluster assumption is the wrong one for a given dataset, only whether the clustering it produced is internally self-consistent.

The predictable mistake. A silhouette score of 0.69, reported without qualification, sounds like a strong, general endorsement of the clustering. It is only ever an endorsement *relative to what the metric can see* — the shape of the clusters it was given, not whether the clustering algorithm's underlying assumption fits the data. Section 3 computed a silhouette score for k-means on the bot/human data too (not shown above, but worth computing as an exercise): it will come out positive and unremarkable-looking, precisely because k-means always produces round-looking clusters by construction, and silhouette is measuring roundness, not correctness.

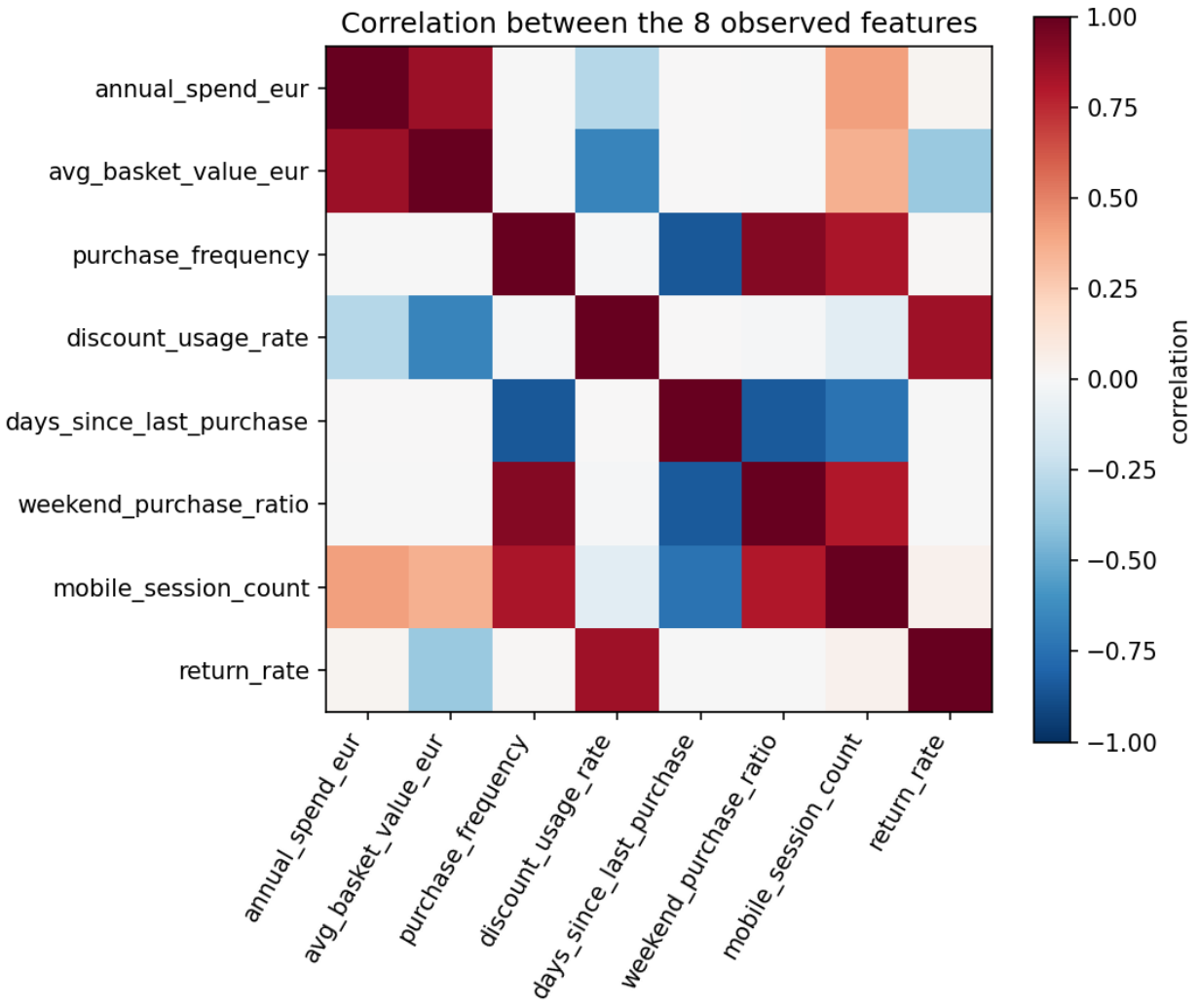
> **Try this:** compute the silhouette score for k-means' $k = 2$ clustering on the bot/human session data, and compare it with the DBSCAN result's equivalent (scikit-learn's `silhouette_score` excludes noise points). Does the internal metric alone give any hint that k-means' ARI is effectively zero?

---

5. Principal component analysis, derived two ways

5.1 The idea, before the algebra

Aurora's account table has eight columns, and the correlation matrix below shows immediately that they are not eight independent pieces of information — spend, basket value and mobile session count all move together, because they all partly reflect the same underlying tendency to spend.



Correlation between the 8 standardised account features. The block of warm cells among spend-related columns, and the separate block among engagement-related columns, is the correlation structure PCA is about to compress.

Principal component analysis (PCA) finds the direction in feature space along which the data varies the *most*, calls it the first principal component, then finds the next-most-varying direction perpendicular to the first, and so on. "Varies the most" stands in for "carries the most information": a direction along which every point looks nearly identical contributes almost nothing to telling points apart, and is a safe direction to discard.

5.2 The variance-maximisation derivation

For data standardised to zero mean and unit variance per column, $X \in \mathbb{R}^{m \times n}$, the first principal component is a unit vector $v \in \mathbb{R}^n$ that maximises the variance of the data projected onto it:

$$\mathrm{Var}(Xv) = \frac{1}{m-1}\|Xv\|^2 = \frac{1}{m-1} v^\top X^\top X v = v^\top \Sigma v, \qquad \Sigma = \frac{1}{m-1} X^\top X$$

where Σ is the sample covariance matrix (for standardised columns, Σ's diagonal is 1 and its off-diagonal entries are correlations). Maximising $v^\top \Sigma v$ subject to $\|v\| = 1$ needs a constraint, since $v^\top \Sigma v$ grows without bound as v scales up — introduce a Lagrange multiplier λ for the constraint:

$$\mathcal{L}(v, \lambda) = v^\top \Sigma v - \lambda(v^\top v - 1)$$

$$\frac{\partial \mathcal{L}}{\partial v} = 2\Sigma v - 2\lambda v = 0 \quad \Longrightarrow \quad \Sigma v = \lambda v$$

The stationary points of the constrained problem are exactly the **eigenvectors of** Σ, with λ the corresponding **eigenvalue**. Substituting back, $v^\top \Sigma v = v^\top(\lambda v) = \lambda \|v\|^2 = \lambda$: the variance captured by direction v *is* its eigenvalue. The direction that maximises variance is therefore the eigenvector with the **largest** eigenvalue; the second component is the eigenvector with the next-largest eigenvalue, subject to being orthogonal to the first (a constraint automatically satisfied, because Σ is symmetric and its eigenvectors are orthogonal by the spectral theorem); and so on. Ordering all n eigenvalues from largest to smallest and keeping their eigenvectors gives every principal component at once.

Worked example. On the two standardised features from section 2 — spend and visit frequency — the covariance matrix is

$$\Sigma = \begin{pmatrix} 1.000 & 0.171 \\ 0.171 & 1.000 \end{pmatrix}$$

For any 2×2 matrix of this symmetric form, $\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$, the eigenvalues are $1+r$ and $1-r$, with eigenvectors $\frac{1}{\sqrt{2}}(1,1)$ and $\frac{1}{\sqrt{2}}(1,-1)$ respectively — a fact checkable directly from $\Sigma v = \lambda v$ without any numerical solver. Substituting $r = 0.171$: eigenvalues 1.171 and 0.829, matching `numpy.linalg.eigh`'s output on this data to three decimal places (the small remaining difference is `numpy.cov`'s sample covariance using $m-1$ in its denominator, so the diagonal of Σ is not exactly 1). The first component points along $(1,1)/\sqrt{2}$ — the "spend and visit often together" direction — which makes sense: customers who spend more tend also to visit more often, so the direction of greatest joint variation is the diagonal, not either axis alone.

5.3 The same answer from the SVD

Forming $\Sigma = X^\top X/(m-1)$ explicitly and finding its eigenvectors is one route to the same components; the **singular value decomposition** of the centred data matrix itself is the numerically preferred one, because it never explicitly computes $X^\top X$ — squaring a matrix squares its condition number, the same ill-conditioning concern lesson 3 raised for the normal equation. For centred X (mean subtracted, not necessarily variance-scaled),

$$X = USV^\top$$

with U and V orthogonal and S diagonal with non-negative entries $s_1 \geq s_2 \geq \ldots$ (the **singular values**). The columns of V are exactly the eigenvectors of $X^\top X$, and

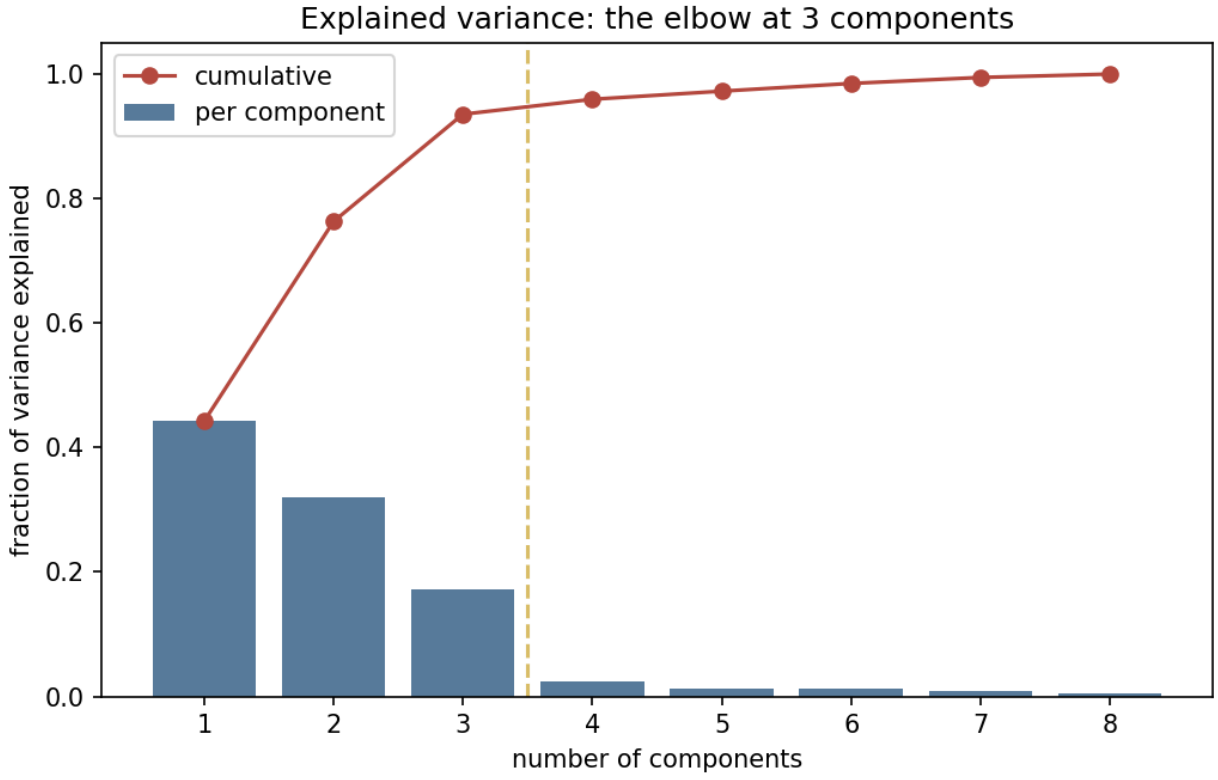
$$X^\top X = VSU^\top USV^\top = VS^2V^\top \quad \Longrightarrow \quad \Sigma v_i = \frac{s_i^2}{m-1} v_i$$

so $\lambda_i = s_i^2/(m-1)$ recovers the eigenvalues exactly, without ever forming Σ. On the 8-feature account table, all three routes — `numpy.linalg.eigh` on the covariance matrix, `numpy.linalg.svd` on the centred data with $\lambda_i = s_i^2/(m-1)$, and `sklearn.decomposition .PCA` — agree to within 5×10^{-15}, floating-point rounding and nothing more.

5.4 How many components: the scree plot

Principal component analysis (PCA) hands back as many components as there were columns, ordered by how much variance each explains. Deciding how many to keep is the one judgement it does not make for you, and the usual instrument is a **scree plot**: the explained variance per component, read for the point where it stops falling steeply.

Component	Eigenvalue	Variance explained	Cumulative
PC1	3.539	44.2%	44.2%
PC2	2.568	32.1%	76.3%
PC3	1.380	17.2%	93.5%
PC4	0.192	2.4%	96.0%
PC5–PC8	0.043–0.104	0.5–1.3% each	100.0%



Per-component and cumulative explained variance for the 8 account features. The first 3 components explain 93.5% of total variance; the 4th adds only 2.4% more — a sharp elbow at 3.

The elbow lands exactly on **3** — the number of latent factors (`retail_data.py`'s `spending_propensity`, `engagement`, `price_sensitivity`) that actually generated all 8 observed columns. This clean an elbow will not appear on every dataset a student meets after this course — it exists here because the generator built the eight columns as noisy linear combinations of exactly three underlying signals, and the scree plot is recovering, from the data alone, a number this lesson chose to withhold from every method until now. The general recipe when the elbow is softer — as it usually is on real data — is to keep enough components to explain a chosen fraction of variance (commonly 90–95%), or to use the components as a cross-validated preprocessing step and let downstream performance decide.

---

6. Reconstruction error, and the anomalies a 2-D plot cannot show

6.1 Reconstructing from a truncated model

Keeping only the top k components and mapping a point back into the original n-dimensional space gives a **reconstruction** $\hat{x}_i$ — the closest point to x_i that lies on the k-dimensional subspace PCA selected. Concretely, if $V_k \in \mathbb{R}^{n \times k}$ holds the first k eigenvectors as columns,

$$\hat{x}_i = V_k V_k^\top x_i$$

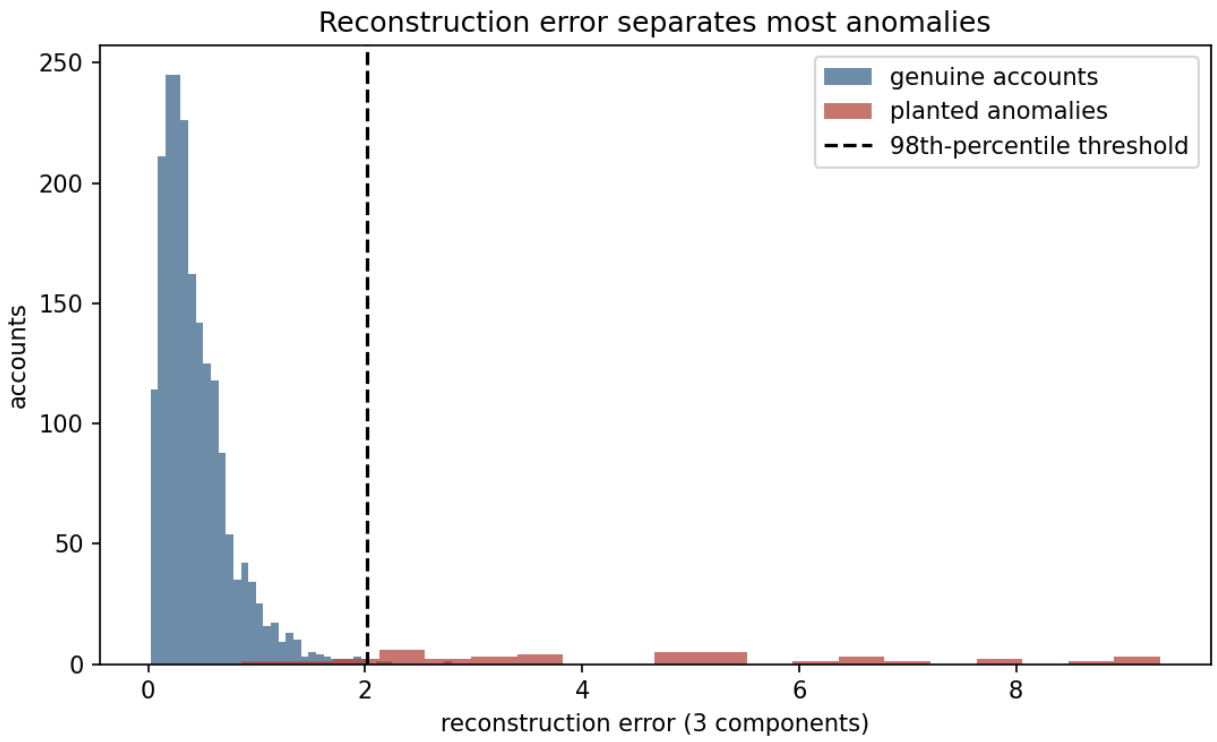
A point that genuinely follows the correlation structure the top k components were fit to reconstructs almost exactly, because that structure is what those components describe. A point that violates it — individually ordinary values that do not co-occur the way genuine points' do — cannot be well approximated by a subspace built for a correlation pattern it does not follow, and its **reconstruction error**

$$e_i = \|x_i - \hat{x}_i\|^2$$

is large. This is precisely the account-takeover scenario `retail_data.py` builds into notebook 03: every value assigned to an anomalous account is drawn from the same marginal range genuine accounts occupy, so no single-column threshold would flag it. What the anomaly violates is the *relationship between* columns — exactly what the three kept components encode, and what the five discarded components would otherwise have had to explain.

Using $k = 3$ components (the number section 5.4 recovered) and flagging the top 2% of accounts by reconstruction error:

	Value
Threshold (98th percentile)	2.019
Accounts flagged	40
Planted anomalies caught	**37 of 40**
Recall / precision	92.5% / 92.5%
Mean reconstruction error, anomalies	4.676
Mean reconstruction error, genuine accounts	0.431



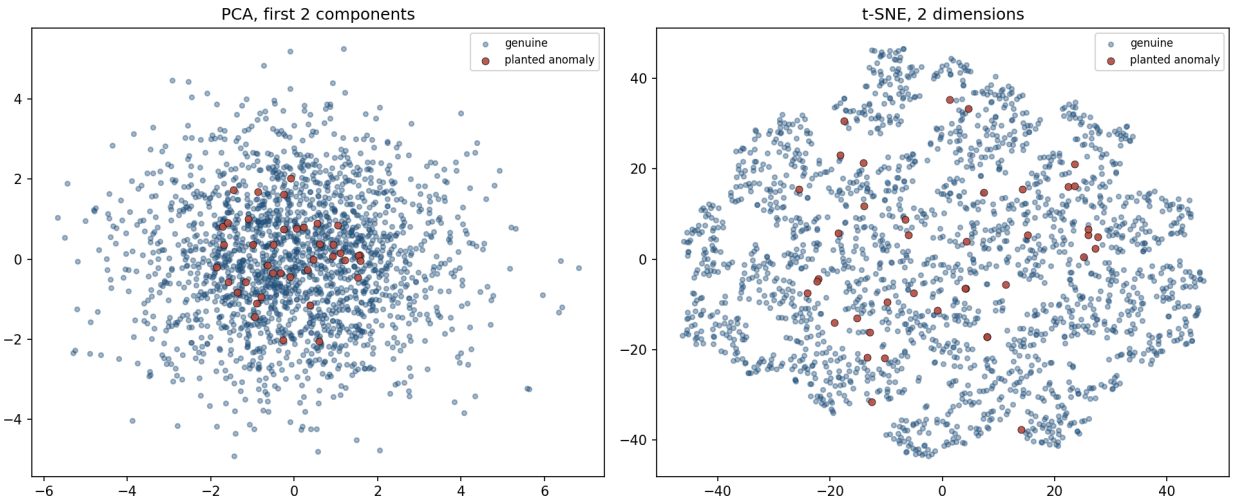
Reconstruction error, genuine accounts against planted anomalies. The two distributions overlap at the low end — some anomalies are unlucky enough to land near the genuine subspace by chance — but the separation is large enough that a threshold near the genuine population's upper tail catches the great majority.

37 of 40 disguised accounts caught, using only the relationship between eight ordinary-looking numbers — this lesson's one number worth carrying forward. Note also what the 8% miss rate means: 3 anomalies happened to land, by chance, close enough to the genuine correlation structure to evade the threshold, and roughly the same number of genuine accounts sit above it purely by chance (40 flagged, 37 correct, 3 false positives) — this is a detection problem with a real error rate, not a guarantee, and should be reported to Aurora's fraud team as such.

How real that error rate is can be measured rather than asserted. Repeating the identical procedure on seven further batches from the same process, the count caught runs from **32 to 38** of 40, a median of 36. Carry the 37 as the memorable figure and the range as what it means: this finds around nine in ten, and *which* nine depends on where a given quarter's fraudsters happen to land.

6.2 Why a 2-D plot is the wrong tool for this particular job

The instinctive next step, having used a 2-D scatter to *see* clustering structure throughout this lesson, is to plot the account data on its first two principal components and look for the anomalies as visual outliers.



Genuine accounts and planted anomalies, projected onto 2 dimensions by PCA (left) and by t-SNE (right, section 7). In neither picture do the anomalies stand out as the accounts farthest from the centre.

Measuring it directly: the mean distance from the genuine-account centroid is **1.30** for anomalies against **2.20** for genuine accounts under the PCA projection, and **22.16** against **30.17** under t-SNE — in *both* embeddings, the anomalies sit **closer** to the centre than genuine accounts do, on average, not farther. This is not an artefact of one random draw: the same ordering holds at two further seeds of the data generator (`Docs/worked_examples.py` checks both), so it is a property of how the anomalies were built, not a coincidence of this run.

The mechanism is direct. A genuine account's eight values move together — high spend tends to come with high basket value and more mobile sessions — which is exactly what pushes it a meaningful distance along a real principal direction. An anomalous account's eight values are drawn independently, each from a plausible range, but with no such co-movement; averaged over eight uncorrelated draws, an anomaly is more likely to land somewhere unremarkable in *any* single projection than to land consistently far out along one particular direction. **The information that identifies these accounts is not "how far from the middle" — the question any 2-D scatter answers — but "how well do the kept components predict the columns that were dropped," a question a 2-D plot of those same kept components cannot ask, because it has already thrown away the comparison.**

The predictable mistake. Section 2 taught, correctly, that a 2-D scatter is often the fastest way to see clustering structure, and the instinct to reach for one here first is not unreasonable — it is the same instinct that worked for choosing k and for judging DBSCAN against k-means throughout this lesson. The failure is treating "plot it and look" as a substitute for a defined measurement, rather than as a first pass that a proper check must follow. Reconstruction error is not more sophisticated for its own sake: it uses the *entire* kept subspace, including the third component visualisation drops, and, implicitly, the discrepancy against everything the visualisation never had at all.

---

7. t-SNE: a tool built for looking, not for measuring

t-SNE (t-distributed stochastic neighbour embedding) is a dimensionality-reduction method built for a different purpose than PCA: not to maximise variance captured, but to arrange points in 2 or 3 dimensions so that points *close together in the original space stay close together in the picture* — with no commitment to preserving distances between points that were originally far apart. It does this by converting distances in the original space into probabilities of being "neighbours," doing the same in the low-dimensional embedding, and moving points in the embedding to make the two sets of probabilities match as closely as possible.

The consequence students meet immediately in notebook 03 is that t-SNE is extremely good at making genuinely separate clusters visually obvious — often more obvious than PCA's first two components manage — precisely because it is optimising for exactly that, at the cost of two properties PCA has and t-SNE does not: **distances between clusters in a t-SNE plot are not meaningful** (two clusters drawn far apart are not necessarily more different than two drawn close together), and **there is no fixed transform** to place a new point into an existing t-SNE embedding the way `pca.transform(x_new)` places a new point into a PCA one — a fresh point requires rerunning the whole optimisation.

The full derivation — the Kullback–Leibler divergence between two probability distributions over pairs of points, and its gradient with respect to every embedded position simultaneously — is beyond this course's scope; it is genuinely more involved than every other method in this lesson; see the further reading for the source. What is worth carrying forward is the *use case* it is suited to: exploratory, human-facing visualisation of cluster structure, not a quantitative distance measurement, and — as section 6.2 showed directly — not a substitute for a proper anomaly check either, since it inherited the exact same blind spot PCA's 2-D projection had.

---

8. Choosing among today's methods

Task	Reach for
Group similar records with no labels, clusters plausibly round	k-means
Group similar records, shape unknown or possibly irregular/nested	DBSCAN
Understand nested structure at multiple scales	Agglomerative clustering
Reduce many correlated numeric columns to a handful, for modelling or storage	PCA
Flag records that violate the relationship between columns, not any one column	PCA reconstruction error
Produce a 2-D picture for a human to look at	t-SNE (or PCA, if a fixed, interpretable transform is also needed)

Summary

- k-means minimises within-cluster sum of squares by alternating an exact assignment step and an exact update step (Lloyd's algorithm); both steps can only lower the objective, and with

finitely many partitions of m points the sequence must converge — to *a* local minimum, not necessarily the global one.

- Initialisation matters concretely: 30 naive random starts on the customer data ranged from WCSS 374.1 (global optimum) to 1,390.4 — 3.7x worse — on identical data with the identical update rule. k-means++ narrows that gap by preferring centres far from ones already chosen.
- The elbow and the silhouette score are two different diagnostics for choosing k; here they agree, both pointing to $k = 4$, matching the number of segments the data was built with.
- On a dataset built as a dense core inside a diffuse ring, k-means and Ward-linkage hierarchical clustering both scored ARI within noise of 0 — not a tuning failure, but a mismatch between what they optimise (compact, round clusters) and the data's actual shape. DBSCAN, which clusters by density rather than shape, recovered the split at **ARI = 0.941** — with an `eps` read off this week's own data, which section 3.4 shows is the part that does not transfer to next week's.
- Internal validation metrics (silhouette) measure whether a clustering is self-consistent; external metrics (ARI) measure agreement with an outside ground truth, which real problems rarely have as cleanly as this lesson's synthetic data does.
- PCA's components are the eigenvectors of the covariance matrix, found either by eigendecomposition or by the numerically preferred SVD; the two routes and scikit-learn's implementation agreed to within 5×10^{-15} on this lesson's 8-feature dataset. The scree plot recovered the data's true latent dimensionality (3) directly from the data.
- **37 of 40 disguised fraudulent accounts were caught by PCA reconstruction error** — accounts that were individually unremarkable on every single column and gave themselves away only in how the columns related to each other. Across seven further batches from the same process the count ran from 32 to 38, a median of 36.
- Neither a PCA nor a t-SNE 2-D scatter separated those same anomalies visually — both placed them, on average, *closer* to the centre than genuine accounts, at more than one random seed. A low-dimensional picture is not a substitute for a measurement that uses the dimensions the picture discarded.

Homework

`Exercises/08_unsupervised_learning.md`, discussed at the start of Lesson 9, **Friday 20 November 2026**.

Notation used in this lesson

Symbol	Meaning
J	k-means' within-cluster sum of squares (WCSS)
r_{ij}	1 if point i is assigned to cluster j, else 0
μ_j	the centre of cluster j
k	number of clusters (context 2–4), or a k-distance neighbour count (context 3.3)
s_i	silhouette score of point i
Σ	sample covariance matrix of standardised data
v, λ	an eigenvector of Σ and its eigenvalue
U, S, V	the singular value decomposition, $X = USV^\top$
$\hat{x}_i$, e_i	PCA reconstruction of point i, and its reconstruction error

Symbol	Meaning
`eps`, `min_samples`	DBSCAN's neighbourhood radius and core-point threshold

---

Further reading

Resource	Type	Why read it
scikit-learn: clustering	Official docs	Every clustering method in this lesson, plus several not covered, with the parameters that matter
scikit-learn: decomposition (PCA)	Official docs	PCA's parameters and variants (kernel PCA, incremental PCA) precisely stated
Ester, Kriegel, Sander & Xu, *A Density-Based Algorithm for Discovering Clusters* (1996)	Paper	The original DBSCAN paper, section 3.3's algorithm from its source
van der Maaten & Hinton, *Visualizing Data using t-SNE* (2008)	Paper	The full KL-divergence derivation section 7 deliberately left out
Hastie, Tibshirani & Friedman, *Elements of Statistical Learning*, ch. 14	Book	Clustering and PCA (section 14.5) in the same rigorous treatment as the rest of this course's further reading
Jolliffe, *Principal Component Analysis*, 2nd ed.	Book	The full theory behind section 5, including the connections to factor analysis this lesson only gestures at